

You are working directly on my locally cloned IDPS repository.

Your job is to inspect the ENTIRE existing codebase first and then optimize and upgrade the system into a technically credible real-time Intrusion Detection and Prevention System.

Do not just improve the UI.

The primary objective is to make the detection, response, firewall enforcement, networking architecture, reliability, security, and testing significantly better while preserving the existing working functionality.

===== MY ACTUAL NETWORK SETUP
=====

I currently do not have a physical router.

My intended real-world lab setup is:

PHONE | | USB cable + USB Tethering ↓ LAPTOP RUNNING IDPS | | Laptop Wi-Fi Hotspot ↓
PROTECTED DEVICES

The laptop will receive Internet from my phone through USB tethering.

The laptop will then share that Internet connection through its Wi-Fi hotspot.

Other devices will connect to the laptop's Wi-Fi hotspot.

Therefore the laptop can act as:

- Internet gateway
- NAT/router
- firewall
- IDPS

The intended traffic path is:

Protected Device ↓ Laptop Wi-Fi interface ↓ IDPS detection ↓ IPS decision ↓ Laptop firewall/routing ↓
USB tethering interface ↓ Phone ↓ Internet

This is the architecture I want the project to support.

IMPORTANT:

Do not assume that I own a physical router.

Do not pretend that simply sniffing packets on my laptop is equivalent to gateway-level prevention.

The system must distinguish between:

HOST MODE

and

GATEWAY MODE

===== 1. FIRST: ANALYZE THE EXISTING CODEBASE
=====

Before making modifications:

1. Inspect every backend file.
2. Inspect every frontend file.
3. Inspect package.json and requirements.
4. Inspect configuration.
5. Inspect the README.
6. Identify the current packet-capture flow.
7. Identify the current detection flow.
8. Identify the current alert/state architecture.
9. Identify the current firewall implementation.
10. Identify the current WebSocket architecture.
11. Identify security weaknesses.
12. Identify incorrect claims in documentation.
13. Identify dead/unreachable code.
14. Identify duplicated logic.
15. Identify race conditions.
16. Identify missing tests.

Do not rewrite everything blindly.

Preserve good existing code where appropriate.

After inspection, create an internal implementation plan and then execute it.

===== 2. DEPLOYMENT MODES
=====

Implement two real deployment modes.

HOST MODE:

Protect the laptop itself.

Traffic:

Internet ↓ Laptop ↓ Host IDPS

The IDPS can detect suspicious traffic affecting the laptop and use the laptop's firewall to protect the laptop.

GATEWAY MODE:

Protect devices behind the laptop.

Traffic:

USB tethering ↓ Laptop WAN interface ↓ Laptop routing/NAT ↓ IDPS ↓ Laptop Wi-Fi hotspot ↓ Protected clients

In GATEWAY mode the laptop is expected to have:

WAN interface = USB tethering

LAN interface = Wi-Fi hotspot

The backend must expose the interfaces separately.

Example configuration:

```
IDPS_MODE=GATEWAY WAN_INTERFACE=<USB_TETHER_INTERFACE>
LAN_INTERFACE=<WIFI_HOTSPOT_INTERFACE>
```

Do NOT hard-code interface names.

The application should detect available interfaces and allow explicit configuration.

```
===== 3. SECURITY MODES
=====
```

Implement:

IDS IPS

IDS:

- capture
- analyze
- detect
- alert
- log

No automatic firewall blocking.

IPS:

- capture
- analyze
- detect
- evaluate policy
- enforce firewall rule
- record result

Dashboard must display:

Deployment Mode: HOST / GATEWAY

Security Mode: IDS / IPS

===== 4. REAL GATEWAY PREVENTION
=====

This is extremely important.

In GATEWAY + IPS mode, the IDPS must NOT simulate prevention by simply ignoring a packet in the Scapy callback.

This:

if source_ip in blocked_ips: return

is NOT real network prevention.

Real prevention must happen through the operating system's firewall/routing layer.

The architecture should be:

Packet Capture ↓ Detection Engine ↓ Response Policy ↓ Firewall Manager ↓ OS Firewall ↓ Packet actually dropped

For gateway traffic, the firewall must be capable of blocking forwarded traffic, not just traffic destined for the laptop itself.

Distinguish:

INPUT traffic

from:

FORWARD traffic.

===== 5. FIREWALL MANAGER =====

Create a dedicated abstraction:

FirewallManager

It should support:

- detect platform
- block source IP
- unblock source IP
- verify block
- list IDPS blocks
- expire blocks
- remove IDPS rules
- protect trusted addresses
- return structured results

Do not place firewall commands inside the detection engine.

Architecture:

Detection Engine ↓ Response Engine ↓ FirewallManager

Support at least Linux properly.

If Windows/macOS support already exists, improve it safely, but prioritize the platform on which the project is actually being developed.

===== 6. LINUX GATEWAY SUPPORT =====

If the local machine is Linux, implement proper gateway support.

The system should understand:

WAN interface LAN interface IP forwarding NAT FORWARD firewall rules

Do not automatically modify networking configuration without explicit user configuration.

Provide a clear setup/check mechanism that verifies:

1. WAN interface exists.
2. LAN interface exists.
3. LAN has an IP address.

4. IP forwarding is enabled.
5. Firewall backend is available.
6. Required firewall capabilities exist.
7. The IDPS can create/remove its own rules.

Create a diagnostic command or endpoint such as:

/api/system/network-check

It should report:

- WAN interface
- LAN interface
- WAN address
- LAN address
- forwarding enabled/disabled
- firewall available/unavailable
- gateway readiness

Do not silently change the host's network configuration.

===== 7. DO NOT ASSUME WI-FI HOTSPOT
CAPABILITY =====

The laptop Wi-Fi adapter may or may not support creating an access point while simultaneously maintaining the required network configuration.

The application should therefore detect/report this instead of pretending it works.

Provide diagnostics such as:

Wi-Fi AP capable: YES/NO

USB tether interface: DETECTED/NOT DETECTED

Gateway ready: YES/NO

This is particularly important for my USB-tethered-phone → laptop-hotspot setup.

===== 8. RATE DETECTION
=====

Replace the current simplistic one-second counter reset with proper sliding-window detection.

Track per source:

- packet count
- SYN count
- UDP count
- ICMP count
- unique ports
- connection attempts

Support:

1 second 5 seconds optionally 30 seconds

Calculate actual rates.

Do not depend on clearing all dictionaries every second.

Use bounded data structures so memory cannot grow indefinitely.

===== 9. SYN FLOOD DETECTION
=====

Improve SYN flood detection.

Track:

source IP source port destination IP destination port

Track TCP state where possible:

SYN SYN/ACK ACK RST

Detect:

- excessive SYN rate
- abnormal half-open connection count
- abnormal SYN/ACK ratio
- SYN scanning patterns

Do not treat every SYN as malicious.

Use:

threshold rate duration confidence

===== 10. PORT SCAN DETECTION
=====

Improve port scan detection.

Detect patterns such as:

TCP SYN scan UDP scan vertical scan horizontal scan

Track:

source IP destination IP destination port protocol time window

Generate structured detections.

===== 11. UDP/ICMP FLOOD DETECTION
=====

Implement proper sliding-window detection for:

UDP floods ICMP floods

Include:

rate duration source destination protocol

Use configurable thresholds.

===== 12. ARP SPOOFING
=====

Improve ARP detection.

Track:

IP → MAC

MAC → IP

Track:

first seen last seen MAC changes duplicate IPs ARP frequency

Classify:

NEW_DEVICE MAC_CHANGED DUPLICATE_IP ARP_SPOOFING_SUSPECTED

Do not automatically call every MAC change confirmed spoofing.

Use confidence.

===== 13. IPv6
=====

Add support for:

IPv4 IPv6 TCP UDP ICMP ICMPv6 ARP IPv6 Neighbor Discovery where practical

Do not break IPv4.

===== 14. DETECTION RULE ARCHITECTURE
=====

Create a rule-based architecture.

Example IDs:

NET-SYN-001 NET-UDP-001 NET-ICMP-001 NET-SCAN-001 NET-ARP-001 NET-DOS-001

Every detection should contain:

rule_id type severity confidence source_ip destination_ip source_port destination_port protocol reason
observed_rate first_seen last_seen

Make adding a new rule easy.

===== 15. ALERT MODEL
=====

Improve the Alert model.

Fields:

id timestamp rule_id type severity confidence source_ip destination_ip source_port destination_port
protocol reason observed_rate first_seen last_seen action action_result status

Actions:

NONE ALERT BLOCK UNBLOCK

Results:

NOT_APPLICABLE SUCCESS FAILED

===== 16. FALSE-POSITIVE CONTROL
=====

Do not immediately block an IP based on one weak signal.

Introduce a response policy.

Example:

LOW confidence: Alert only.

MEDIUM confidence: Alert + monitor.

HIGH confidence: Alert + optional block.

CRITICAL confidence: Block if IPS policy allows.

Make this configurable.

===== 17. BLOCK TTL
=====

Every automatic block must have expiration.

Store:

IP rule reason confidence created_at expires_at action result

Example:

Block for 600 seconds.

After expiration:

Firewall rule removed.

Dashboard should show:

Blocked Expires in: 09:42

===== 18. PROTECTED/TRUSTED ADDRESSES
=====

Never automatically block critical local addresses.

Protect:

127.0.0.0/8 ::1

local interface addresses

configured gateway address

configured trusted IPs

Create:

TRUSTED_IPS

configuration.

Before blocking:

validate IP check trusted list check local address check safety rules

Then allow/block.

===== 19. IP VALIDATION
=====

Use Python's ipaddress module.

Reject malformed values.

Never interpolate raw user input into shell commands.

Never use:

shell=True

for firewall operations.

Use subprocess argument arrays.

===== 20. STATE MANAGEMENT =====

The existing project has shared global state.

Improve this.

Scapy capture thread FastAPI WebSocket

must not freely mutate the same dictionaries without synchronization.

Use:

thread-safe state

or:

event queue

Preferred:

Packet Capture ↓ Event Queue ↓ Detection Worker ↓ State Store ↓ Response Engine ↓ Firewall Manager
↓ WebSocket Publisher

Keep the architecture reasonably simple.

Do not turn this into an unnecessarily complex distributed system.

===== 21. WEBSOCKET =====

Create a proper WebSocket ConnectionManager.

Support:

connect disconnect broadcast

Use event-based updates where practical.

Events:

ALERT_CREATED IP_BLOCKED IP_UNBLOCKED DEVICE_DISCOVERED METRIC_UPDATE ENGINE_STATUS

Do not continuously send huge amounts of unnecessary data.

===== 22. API SECURITY =====

Protect administrative APIs.

Especially:

POST /block/{ip} POST /unblock/{ip} POST /scan

and future configuration APIs.

Add authentication and role-based authorization.

Roles:

viewer analyst administrator

Only authorized users can:

block unblock scan change thresholds change IDS/IPS change deployment mode

===== 23. CORS =====

Remove wildcard CORS.

Do not use:

allow_origins=["*"]

with credentials.

Use configuration.

===== 24. CONFIGURATION =====

Use environment variables/configuration.

Example:

IDPS_DEPLOYMENT_MODE=HOST IDPS_SECURITY_MODE=IDS

WAN_INTERFACE= LAN_INTERFACE=

SYN_FLOOD_THRESHOLD=100 UDP_FLOOD_THRESHOLD=200 ICMP_FLOOD_THRESHOLD=100
PORT_SCAN_THRESHOLD=10 SUSPICIOUS_RATE_THRESHOLD=500

BLOCK_TTL_SECONDS=600

TRUSTED_IPS=

ALLOWED_ORIGINS=

Do not hard-code:

wlp1s0

or any other interface.

===== 25. NETWORK DIAGNOSTICS
=====

Add a system/network diagnostics feature.

The backend should be able to report:

Interfaces IP addresses Default route Gateway USB tether detection Wi-Fi AP capability where detectable IP forwarding Firewall backend Current deployment mode Gateway readiness

The dashboard should show this clearly.

For example:

NETWORK STATUS

USB Tethering CONNECTED

WAN usb0

Wi-Fi Hotspot ACTIVE

LAN 192.168.50.1/24

IP Forwarding ENABLED

Firewall READY

Gateway READY

===== 26. FRONTEND
=====

Do not spend most of the effort on visual effects.

Turn the dashboard into a functional SOC interface.

Sections:

Overview Detections Network Devices Response Configuration Diagnostics

Show:

Deployment Mode Security Mode Engine Status WAN LAN Firewall status Blocked IPs Active alerts

For every alert show:

source destination protocol rule confidence rate reason action firewall result expiration

===== 27. HOST MODE USER EXPERIENCE
=====

Clearly display:

HOST MODE

"Protecting this machine."

Do not claim network-wide protection.

===== 28. GATEWAY MODE USER EXPERIENCE
=====

Clearly display:

GATEWAY MODE

"Protecting devices connected behind this machine."

Show:

WAN interface LAN interface forwarding status firewall status

If gateway prerequisites are not satisfied:

Gateway readiness: NOT READY

and explain why.

Do not automatically enable routing just because the user selects GATEWAY mode.

===== 29. TESTING
=====

Create a real pytest suite.

Test:

SYN flood UDP flood ICMP flood port scanning ARP spoof detection IPv6 alert deduplication rate windows
block expiration firewall manager IP validation IDS mode IPS mode HOST mode GATEWAY mode API
authorization WebSocket behavior

Test:

below threshold threshold above threshold

Also test false positives.

===== 30. FIREWALL TEST MODE
=====

Because this application can modify real firewall rules, implement a safe simulation mode.

Example:

FIREWALL_MODE=SIMULATED

In simulated mode:

Detection works normally.

FirewallManager records:

BLOCK REQUESTED

but does not change the actual OS firewall.

Support:

FIREWALL_MODE=REAL

only when explicitly enabled.

This will make development and testing much safer.

===== 31. LOGGING =====

Replace important print() calls with structured logging.

Log:

packet capture startup packet capture failure detection alert block request block success block failure
unblock network diagnostic failures authentication failures configuration changes

Do not log sensitive information unnecessarily.

===== 32. README =====

Update README to accurately describe the actual architecture.

Include:

1. Project overview.
2. Architecture diagram.
3. HOST mode.
4. GATEWAY mode.
5. IDS mode.
6. IPS mode.
7. USB tethering → laptop → Wi-Fi hotspot setup.
8. Network requirements.
9. Firewall requirements.
10. Linux setup.
11. Diagnostics.
12. Detection rules.
13. Testing.
14. Limitations.

Explicitly explain:

"My laptop can act as the gateway when it receives Internet through USB tethering and shares the connection through its Wi-Fi hotspot."

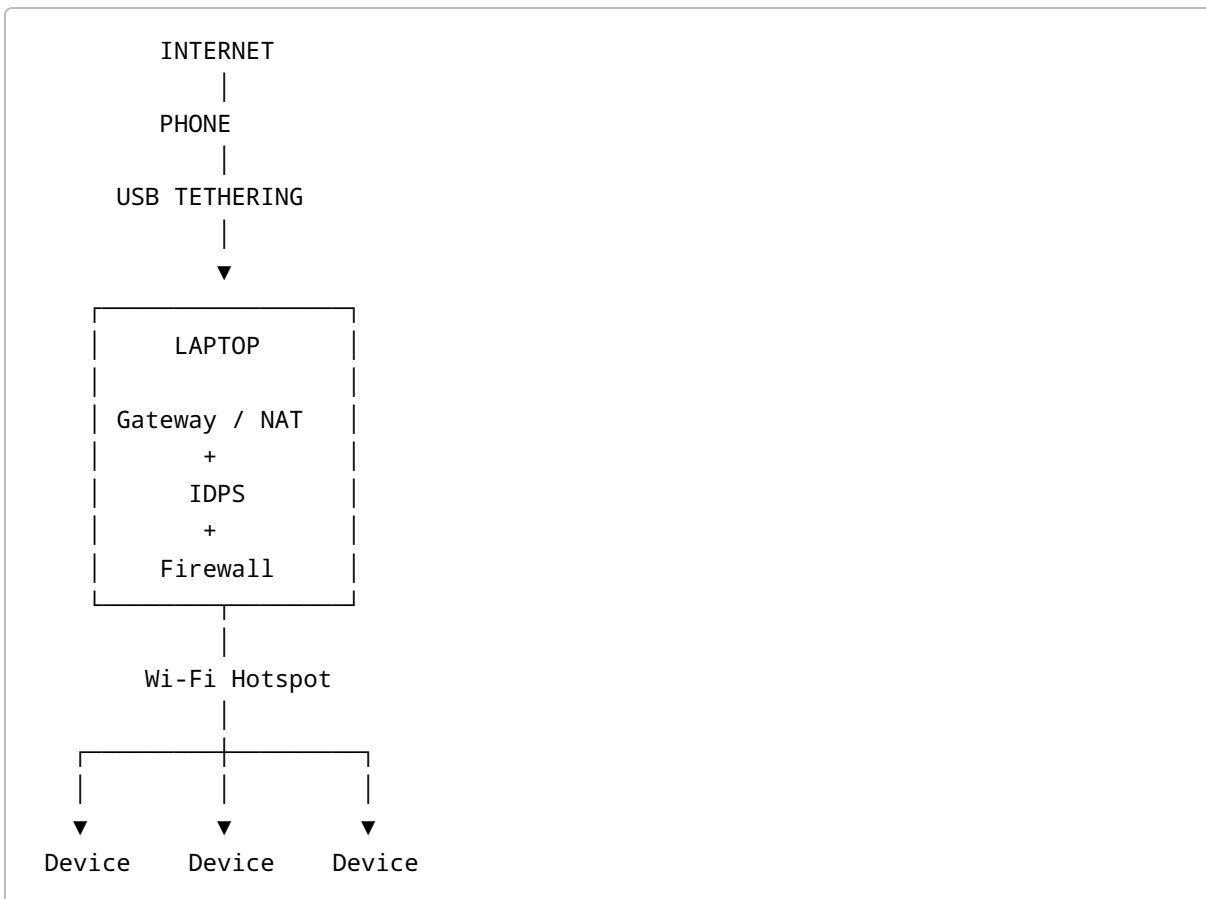
Also explain:

"The IDPS can provide real network-level prevention only when traffic from protected devices actually passes through the laptop."

Do not claim that packet analysis alone provides prevention.

===== 33. DOCUMENT THE EXACT LAB TOPOLOGY
=====

Add this conceptual architecture to the documentation:



Explain that the laptop is the security enforcement point.

===== 34. PERFORMANCE
=====

Do not let packet analysis consume excessive CPU.

Optimize:

packet processing state cleanup WebSocket updates alert storage flow tracking

Use bounded collections.

Avoid repeatedly scanning large alert lists.

===== 35. DO NOT OVERENGINEER
=====

This is a cybersecurity portfolio/research project.

Do NOT introduce:

Kubernetes microservices Kafka Redis clusters distributed databases unnecessary cloud infrastructure

unless genuinely required.

Keep:

FastAPI Scapy Python React TypeScript

and make those components solid.

===== 36. IMPORTANT: PRESERVE FUNCTIONALITY
=====

Do not remove:

existing dashboard existing WebSocket functionality existing detection types existing manual block/unblock
functionality

Improve them.

If a current feature is technically incorrect, fix the implementation rather than simply deleting it.

===== 37. FINAL VERIFICATION
=====

After implementing:

1. Run backend tests.
2. Run frontend tests/build.
3. Check Python syntax.
4. Check TypeScript compilation.
5. Check linting where available.
6. Start backend.
7. Verify REST API.
8. Verify WebSocket.

9. Verify detection engine.
10. Verify simulated firewall mode.
11. Verify real firewall manager carefully.
12. Verify HOST mode.
13. Verify GATEWAY diagnostics.
14. Verify that gateway mode refuses to claim readiness when prerequisites are missing.

===== 38. FINAL REPORT
=====

When finished, give me a concise but technical report containing:

1. What was wrong with the original implementation.
2. What you changed.
3. Files changed.
4. New architecture.
5. Detection improvements.
6. Firewall improvements.
7. HOST mode behavior.
8. GATEWAY mode behavior.
9. USB tethering compatibility.
10. Wi-Fi hotspot requirements.
11. Security improvements.
12. Tests performed.
13. Remaining limitations.

Also provide exact commands/configuration needed to run:

- A. HOST + IDS
- B. HOST + IPS
- C. GATEWAY + IDS
- D. GATEWAY + IPS

For GATEWAY mode, clearly explain which interface is WAN (USB tethering) and which interface is LAN (Wi-Fi hotspot).

Do not claim that GATEWAY mode is working unless the actual network diagnostics confirm that the laptop is configured as a gateway.

The final implementation should be a technically honest, robust IDPS that can run on my laptop today in HOST mode and can operate as a real gateway/IPS when I configure the laptop to share the USB-tethered Internet connection through its Wi-Fi hotspot.